Online Booking System – Deliverable 2

Evidence Report: Knowledge Graph and RAG

Itumeleng Ntela – 43528309

1. Project Overview

The application is an online booking system built as microservices. A browser frontend talks only to an API Gateway, which validates JWT tokens, rate-limits login attempts and routes requests, using the Eureka service registry, to three business services: user-service, resource-service. Each service owns its own PostgreSQL database, and everything runs as separate containers with Docker Compose.

Phase 2 adds a fifth service, soc-service, that turns the system into something a SOC can monitor:

- SOC extension: the gateway and the business services publish structured security and operational events to soc-service, which stores them in its own database
- Detection logic and alerts: a rule engine evaluates four detection rules and raises alerts that are stored, displayed on the frontend's 'SOC / Knowledge Graph' tab and linked to the events that caused them.
- Knowledge graph: events, alerts, application structure and security knowledge are written to a Neo4j graph.
- RAG interface: a natural-language question box answers security questions by retrieving evidence from the graph with Cypher and generating the answer from it with templates

2. Updated Architecture

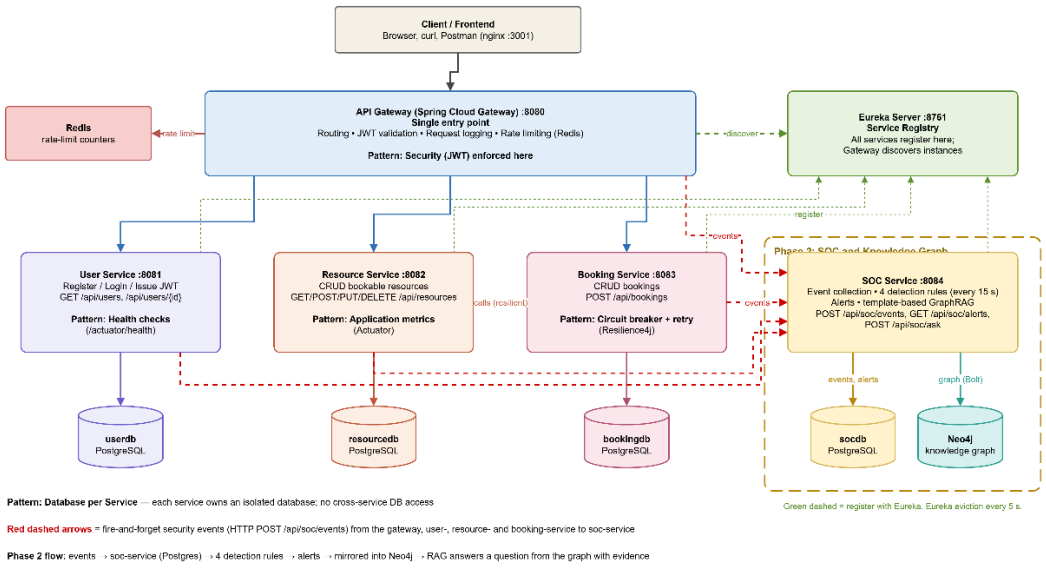


Figure 1 - Updated architecture: API Gateway, microservices, databases, SOC/event collection layer, detection logic and alert service, Neo4j knowledge graph, RAG query interface and the dashboard.

Component	Technology	Role
API Gateway :8080	Spring Cloud Gateway, Redis	Single entry point; JWT validation; rate limiting; request logging; publishes request and 401/403 events
Eureka :8761	Spring Cloud Netflix Eureka	Service registry/discovery (routes use lb://service-name)
user / resource / booking services	Spring Boot, JPA, PostgreSQL (one DB each)	Business logic; each publishes security and business events to soc-service
soc-service :8084	Spring Boot, JPA, Neo4j Java driver 5.24	Event ingestion, 4 detection rules, alert service, graph writer, RAG endpoint
soc-db	PostgreSQL 16	Stores events and alerts (Database-per-Service)
Neo4j :7474 / :7687	Neo4j 5.24 community	Knowledge graph
Frontend :3001	HTML/JS on nginx	Dashboard: alerts table, recent events, RAG question box

Design choices:

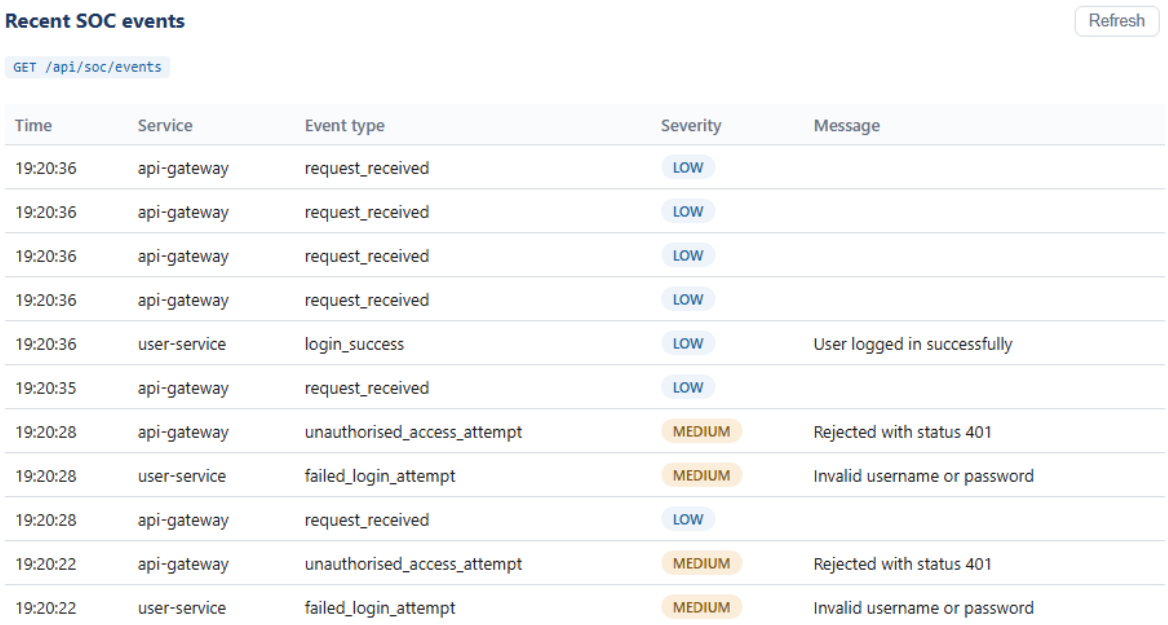
1. a separate SOC microservice keeps monitoring out of the business services and follows the same conventions as them;
2. event publishing is fire-and-forget with a 2-second timeout, so a slow or unavailable SOC can never slow or break a user request;
3. raw Cypher through the Neo4j driver (no ORM) makes every graph write and every RAG retrieval explicit and explainable;
4. All graph writes use MERGE, so they are idempotent and restarts never duplicate data.

3. SOC Event Collection

The services already wrote structured 'event=' log lines in Phase 1. In Phase 2 each service also posts the same event as JSON to POST /api/soc/events on soc-service, alongside the existing logging. soc-service additionally polls every service's health endpoint every 30 seconds.

Source	Events collected	Where it is published
API Gateway	request_received (every request); unauthorised_access_attempt (every 401/403)	LoggingGlobalFilter
user-service	login_success; failed_login_attempt	AuthController
resource-service	resource_created; resource_availability_changed	ResourceController
booking-service	booking_created; dependency_failure (circuit-breaker fallback)	BookingController; ResourceServiceClient
soc-service HealthPoller	service_health_check for api-gateway, user-, resource- and booking-service (status from /actuator/health)	HealthPoller (every 30 s)

Each event is stored in the security_event table of socdb with the specification's fields: eventId, timestamp, serviceName, eventType, severity, userId, sourceIp, endpoint, httpMethod, statusCode, message, correlationId, affectedEntity



4. Detection Rules and Alerts

RuleEngine runs every 15 seconds over unprocessed events. When a rule's condition is met, it raises an alert and marks the matched events as processed so the same burst isn't alerted twice, while a new burst triggers a new alert. Thresholds, windows and severities are configurable in application.yml.

Rule name	Detection logic	Severity	Alert created
Multiple Failed Logins	>= 5 failed_login_attempt events for the same user within 5 minutes	High	Credential attack alert
Unauthorised Endpoint Access	>= 3 unauthorised/invalid-token requests (401/403) from the same user or IP within 5 minutes; >= 6 escalates	Medium / High	Access violation alert
Abnormal Request Rate	>= 20 request_received events to one service within 1 minute	Medium	Possible denial-of-service alert
Service Failure	>= 3 dependency_failure or failed health-check events (status >= 500 / unreachable) for one service within 5 minutes	High	Service failure alert

Rule documentation

Rule	Purpose	Input events	Recommended response
Multiple Failed Logins	Detect credential guessing / stuffing	failed_login_attempt	Lock the account, force a password reset, verify whether it is compromised
Unauthorised Endpoint Access	Detect stolen/expired tokens and access-control probing	unauthorised_access_attempt, invalid_token, missing_or_malformed_token	Review access logs, confirm token compromise, consider blocking the source
Abnormal Request Rate	Detect a possible denial of service	request_received	Inspect source IPs, tighten the gateway rate limiter
Service Failure	Detect dependency / cascading failure	dependency_failure, service_health_check	Check container logs and dependencies; the circuit breaker should contain it

Alerts

Alerts are stored in the alert table of socdb with: alertId, timestamp, ruleName, severity, status, description, relatedEvents, affectedService, affectedUser, affectedEntity and recommendedAction. They are shown on the SOC tab and returned by GET /api/soc/alerts, and each is mirrored to Neo4j. Alerts generated during the demonstration:

Alert ID	Rule	Severity	Description (from system)	Linked events
alert-aa43a2c9-53d5-4827-aaf1-24be11299242	Multiple Failed Logins	HIGH	5 failed login attempts for user 'Tumi' within 5 minute(s).	5
alert-b7b82fb5-c359-4b5e-ad6b-a0fc3e12e0f3	Unauthorised Endpoint Access	MEDIUM	3 unauthorised/invalid-token requests from '172.18.0.1' within 5 minute(s).	3
alert-a0d158ca-a59d-48c9-b606-a14adc7f5da0	Unauthorised Endpoint Access	HIGH	13 unauthorised/invalid-token requests from '172.18.0.1' within 5 minute(s).	13
alert-dc6bf16b-e07a-442e-81a4-896c97f76f54	Service Failure	HIGH	3 failures/unhealthy checks for 'resource-service' within 5 minute(s).	3

The Unauthorised Endpoint Access rule fired twice at different severities, showing the escalation logic. Each alert's relatedEvents list gives traceability back to the original microservice events.

Open alerts Refresh

`GET /api/soc/alerts`

Raised automatically by the detection rules in soc-service once enough matching events have been collected (e.g. 5 failed logins in 5 minutes).

Alert	Rule	Severity	Description	Recommended action
alert-fd6bf8a9-a6f4-40c9-aebb-fd79449dfd3f	Unauthorised Endpoint Access	HIGH	11 unauthorised/invalid-token requests from '172.18.0.1' within 5 minute(s).	Review access logs for this user/IP, confirm whether the token/credentials were stolen, and consider blocking the source.
alert-314a88c5-b521-40fc-8cb3-ad6c887234c2	Multiple Failed Logins	HIGH	11 failed login attempts for user 'Itumeleng' within 5 minute(s).	Temporarily lock the account, prompt a password reset and verify whether the account is compromised.
alert-2ba6ef32-a63a-4047-a088-be13436fdfbf	Unauthorised Endpoint Access	HIGH	6 unauthorised/invalid-token requests from '172.18.0.1' within 5 minute(s).	Review access logs for this user/IP, confirm whether the token/credentials were stolen, and consider blocking the source.
alert-0bc87752-d132-4ed4-9a5b-4e9a6e320ddf	Multiple Failed Logins	HIGH	5 failed login attempts for user 'Itumeleng' within 5 minute(s).	Temporarily lock the account, prompt a password reset and verify whether the account is compromised.
alert-9a49bf40-42c9-45ce-b660-a476586957d9	Unauthorised Endpoint Access	MEDIUM	3 unauthorised/invalid-token requests from '172.18.0.1' within 5 minute(s).	Review access logs for this user/IP, confirm whether the token/credentials were stolen, and consider blocking the source.
alert-8a64389c-cfd5-4818-baa6-eb247f305082	Multiple Failed Logins	HIGH	6 failed login attempts for user 'Philani' within 5 minute(s).	Temporarily lock the account, prompt a password reset and verify whether the account is compromised.
alert-35c0555d-bff3-40ef-b592-511ce44f9616	Unauthorised Endpoint Access	MEDIUM	3 unauthorised/invalid-token requests from '172.18.0.1' within 5 minute(s).	Review access logs for this user/IP, confirm whether the token/credentials were stolen, and consider blocking the source.
alert-e77d27c3-09e2-4f91-be6b-e41f0b82cfe0	Abnormal Request Rate	MEDIUM	60 requests to 'api-gateway' within 1 minute(s) - possible denial-of-service.	Inspect the source IP distribution of recent requests; consider tightening the gateway's rate limiter for this route.
alert-4d960799-6a61-422a-8a3e-07e26fa5cde2	Unauthorised Endpoint Access	HIGH	60 unauthorised/invalid-token requests from '172.18.0.1' within 5 minute(s).	Review access logs for this user/IP, confirm whether the token/credentials were stolen, and consider blocking the source.
alert-dc6bf16b-e07a-442e-81a4-896c97f76f54	Service Failure	HIGH	3 failures/unhealthy checks for 'resource-service' within 5 minute(s).	Check the service's container logs and dependencies; the circuit breaker should already be preventing cascading failure.

5. Knowledge Graph Model

Neo4j is written to directly with Cypher through the Neo4j Java driver. Three data sources are used:

- Application structure from Phase 1: the services, key endpoints and booking-service DEPENDS_ON resource-service, seeded by GraphSeeder at startup.
- Events and alerts generated in Phase 2: written continuously at runtime
- Security knowledge from formal documentation: four Threat nodes tagged with their MITRE ATT&CK technique, six Control nodes with MITIGATES relationships, and a Document node for MITRE T1110 Brute Force linked by DESCRIBES and SUPPORTS_RESPONSE_TO.

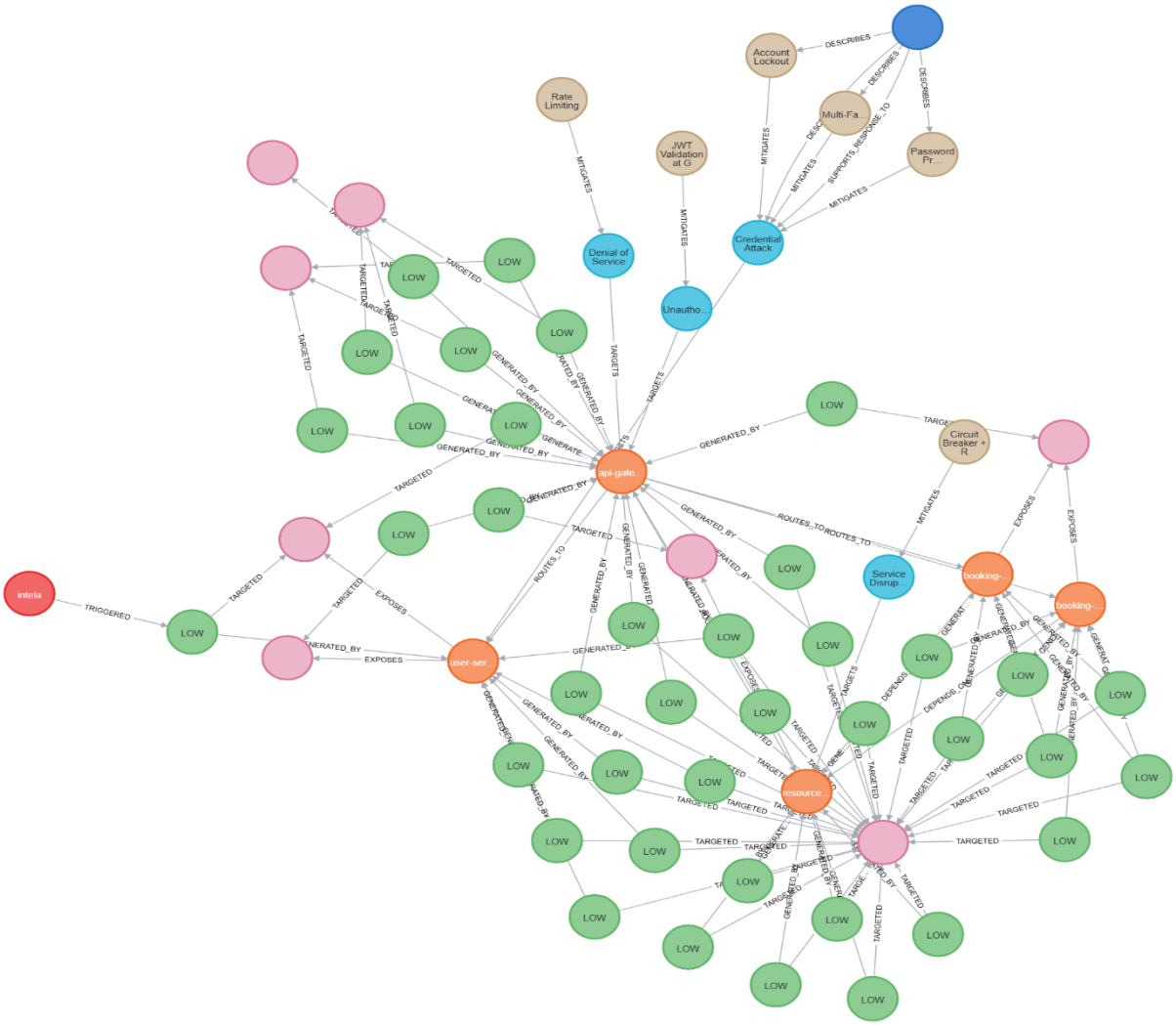
Node labels and key properties

Node label	Key properties	Created by
User	username	GraphWriter.writeEvent
Service	name	GraphSeeder, GraphWriter
Endpoint	path	GraphSeeder, GraphWriter
Event	eventId, timestamp, eventType, severity, message, statusCode	GraphWriter.writeEvent
Alert	alertId, timestamp, ruleName, severity, status, description, recommendedAction	GraphWriter.writeAlert
Threat	name, mitreTechnique, mitreUrl	GraphSeeder
Control	name, description	GraphSeeder
Document	title, url, summary	GraphSeeder

Relationship types

Relationship	Pattern	Meaning
TRIGGERED	(User)->(Event)	The user caused the event
GENERATED_BY	(Event)->(Service)	Service that emitted the event
TARGETED	(Event)->(Endpoint)	Endpoint the event concerned
CREATED_ALERT	(Event)->(Alert)	Events that caused the alert
INDICATES	(Alert)->(Threat)	Threat the alert points to
AFFECTS	(Alert)->(Service)	Service affected by the alert
TARGETS	(Threat)->(Service)	Services exposed to the threat
MITIGATES	(Control)->(Threat)	Control that reduces the threat
DEPENDS_ON	(Service)->(Service)	booking-service depends on resource-service
EXPOSES	(Service)->(Endpoint)	Endpoints each service offers
DESCRIBES	(Document)->(Threat | Control)	Document documents the threat or control
SUPPORTS_RESPONSE_TO	(Document)->(Threat)	Document supplies a response procedure

Evidence: graph visualisation



Evidence: node labels and relationship types

```
neo4j$ CALL db.labels()
```

Table | Text | Code

	label
1	"Event"
2	"Service"
3	"Endpoint"
4	"Threat"
5	"Control"
6	"Document"

```
neo4j$ CALL db.relationshipTypes()
```

Table | Text | Code

	relationshipType
1	"GENERATED_BY"
2	"TARGETED"
3	"ROUTES_TO"
4	"DEPENDS_ON"
5	"EXPOSES"
6	"MITIGATES"

Evidence: an alert linked to a user, a service, a threat and a control

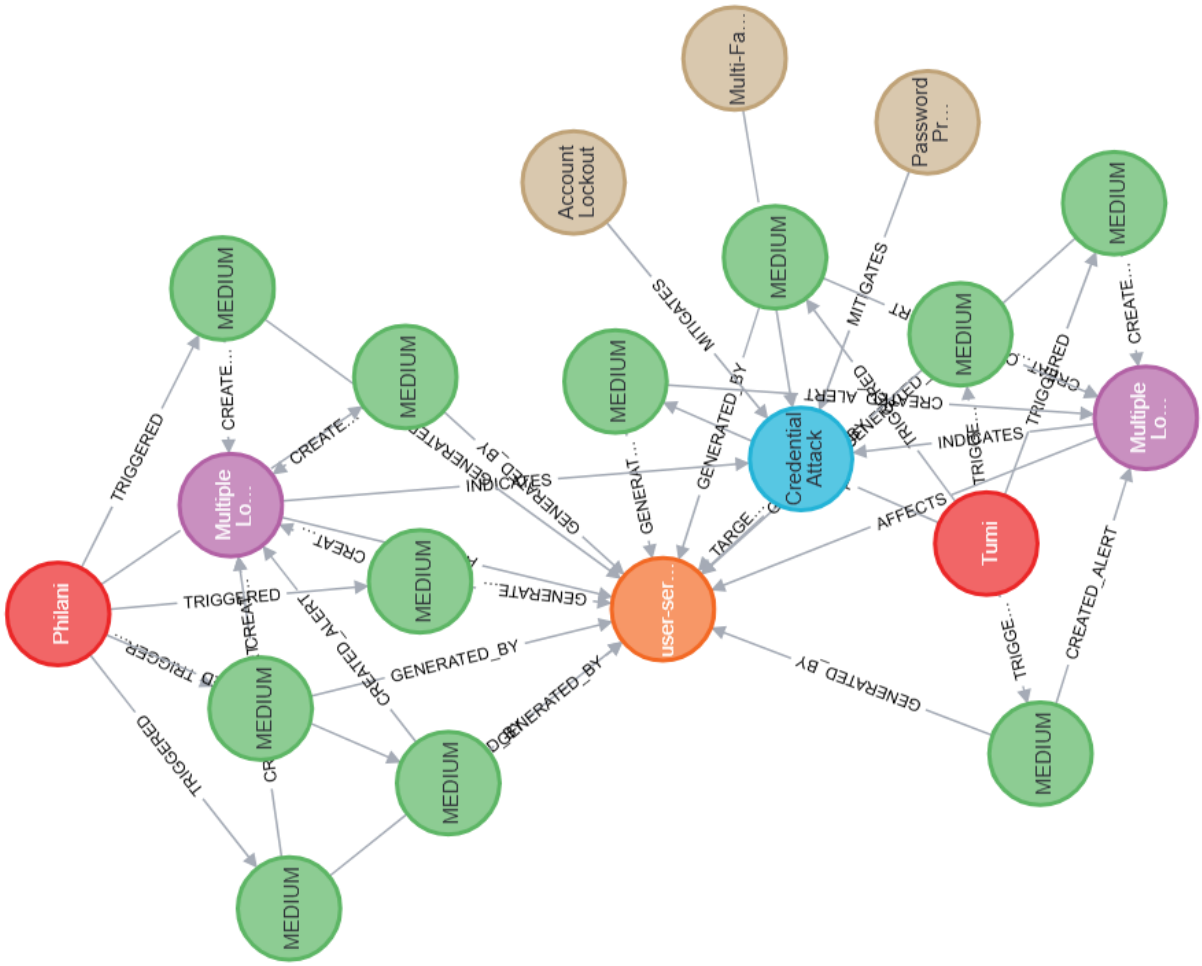
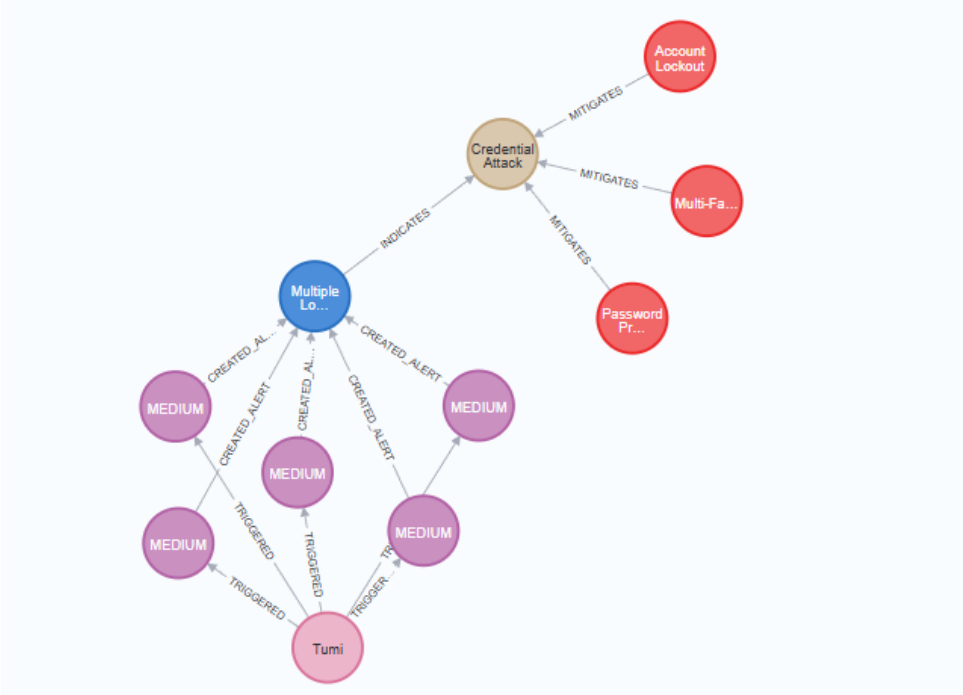


Figure 3 - Traceability chain from an earlier run: user 'Tumi' TRIGGERED five failed-login events which CREATED_ALERT the 'Multiple Failed Logins' alert; the alert INDICATES the Credential Attack threat, which is mitigated by Account Lockout, Multi-Factor Authentication and Password Reset Procedure.



6. Graph Queries Demonstrated

Six Cypher queries were run in Neo4j Browser (the specification requires at least three). Each is also the basis of a RAG question in section 7.

Query 1 - All alerts affecting a service

MATCH (a:Alert)-[:AFFECTS]->(s:Service) RETURN a.alertId, a.ruleName, a.severity, s.name

```
neo4j$ MATCH (a:Alert)-[:AFFECTS]→(s:Service) RETURN a.alertId, a.ru…
```

Table | Text | Code

a.alertId	a.ruleName	a.severity	s.name
"alert-aa43a2c9-53d5-4827-aaf1-24be11299242"	"Multiple Failed Logins"	"HIGH"	"user-service"
"alert-314a88c5-b521-40fc-8cb3-ad6c887234c2"	"Multiple Failed Logins"	"HIGH"	"user-service"
"alert-0bc87752-d132-4ed4-9a5b-4e9a6e320ddf"	"Multiple Failed Logins"	"HIGH"	"user-service"
"alert-8a64389c-cfd5-4818-baa6-eb247f305082"	"Multiple Failed Logins"	"HIGH"	"user-service"
"alert-dc6bf16b-e07a-442e-81a4-896c97f76f54"	"Service Failure"	"HIGH"	"resource-service"

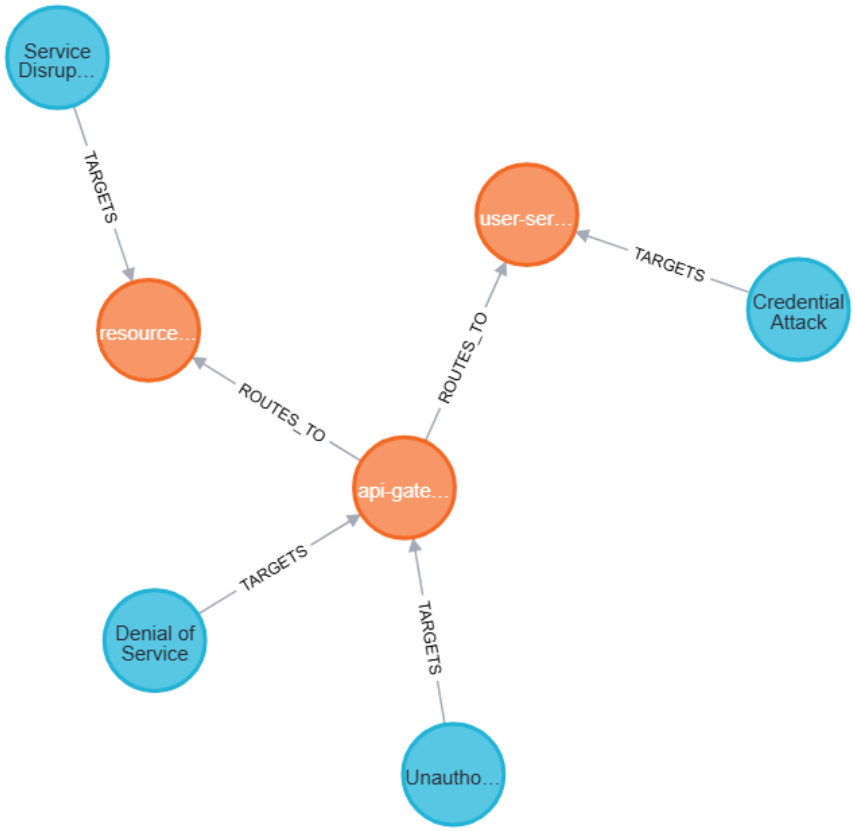
Query 2 - Users linked to high-severity alerts

MATCH (u:User)-[:TRIGGERED]->(:Event)-[:CREATED_ALERT]->(a:Alert {severity:'HIGH'})
RETURN DISTINCT u.username, a.alertId, a.ruleName



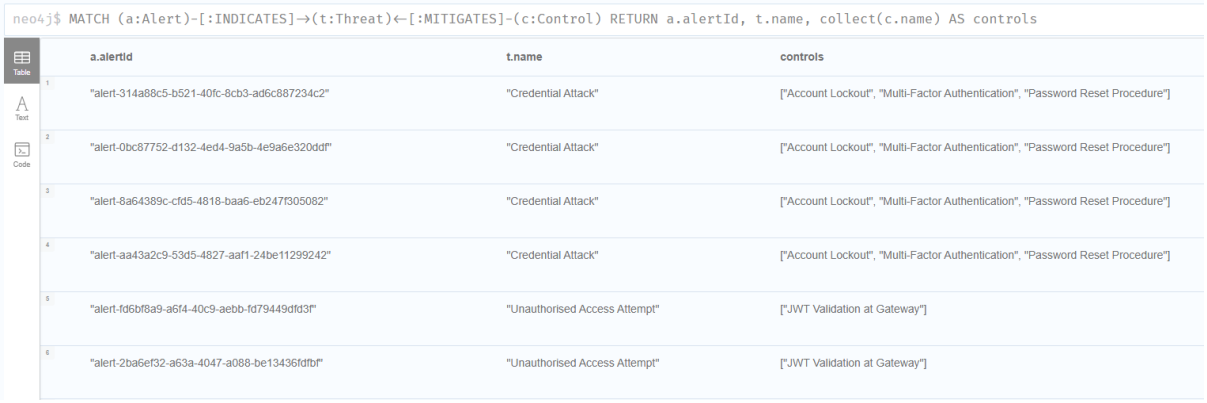
Query 3 - Threats linked to a service

MATCH (t:Threat)-[:TARGETS]->(s:Service) RETURN t, s



Query 4 - Controls for an alert

MATCH (a:Alert)-[:INDICATES]->(t:Threat)<-[:MITIGATES]-(c:Control) RETURN a.alertId, t.name, collect(c.name) AS controls



```
neo4j$ MATCH (a:Alert)-[:INDICATES]→(t:Threat)←[:MITIGATES]-(c:Control) RETURN a.alertId, t.name, collect(c.name) AS controls
```

	a.alertId	t.name	controls
1	"alert-314a88c5-b521-40fc-8cb3-ad6c887234c2"	"Credential Attack"	["Account Lockout", "Multi-Factor Authentication", "Password Reset Procedure"]
2	"alert-0bc87752-d132-4ed4-9a5b-4e9a6e320ddf"	"Credential Attack"	["Account Lockout", "Multi-Factor Authentication", "Password Reset Procedure"]
3	"alert-8a64389c-cfd5-4818-baa6-eb247f305082"	"Credential Attack"	["Account Lockout", "Multi-Factor Authentication", "Password Reset Procedure"]
4	"alert-aa43a2c9-53d5-4827-aaf1-24be11299242"	"Credential Attack"	["Account Lockout", "Multi-Factor Authentication", "Password Reset Procedure"]
5	"alert-fd6bf8a9-a6f4-40c9-aebb-fd79449dfd3f"	"Unauthorised Access Attempt"	["JWT Validation at Gateway"]
6	"alert-2ba6ef32-a63a-4047-a088-be13436fdfbf"	"Unauthorised Access Attempt"	["JWT Validation at Gateway"]

Query 5 - Dependency impact

MATCH (d:Service)-[:DEPENDS_ON]->(s:Service {name:'resource-service'}) RETURN d.name AS affectedService

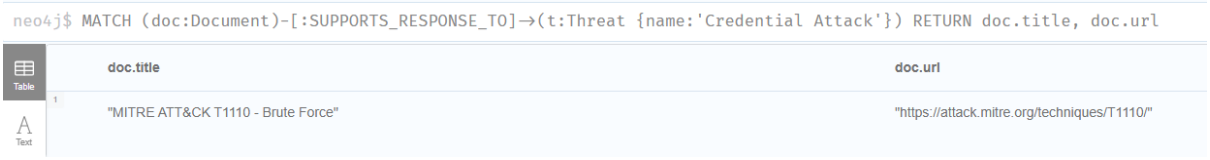


```
neo4j$ MATCH (d:Service)-[:DEPENDS_ON]→(s:Service {name:'resource-service'}) RETURN d.name AS affectedService
```

	affectedService
1	"booking-service"
2	"booking-service"

Query 6 - Response procedure for a threat

MATCH (doc:Document)-[:SUPPORTS_RESPONSE_TO]->(t:Threat {name:'Credential Attack'}) RETURN doc.title, doc.url



```
neo4j$ MATCH (doc:Document)-[:SUPPORTS_RESPONSE_TO]→(t:Threat {name:'Credential Attack'}) RETURN doc.title, doc.url
```

	doc.title	doc.url
1	"MITRE ATT&CK T1110 - Brute Force"	"https://attack.mitre.org/techniques/T1110/"

7. RAG Interface

How the RAG component works

The system uses template-based GraphRAG, which is Option C in the specification, built on Cypher-based retrieval (Option A style). No LLM and no vector search is used. The retrieval process, which can be traced end to end in RagService, is:

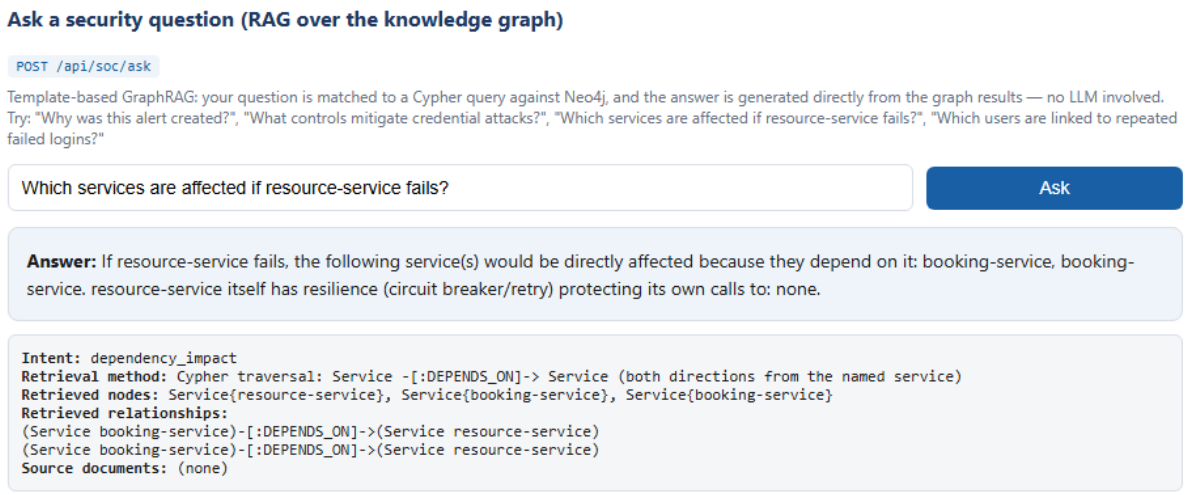
1. The user types a natural-language question in the SOC tab (POST /api/soc/ask).
2. Keyword rules map the question to one of eight intents Entities such as a service name, a threat name or an alert ID are extracted from the text.

3. The intent's Cypher query runs against Neo4j The returned nodes and relationships are the retrieved evidence.
4. A fixed sentence template is filled with the retrieved values to generate the answer, so the answer can only state what the graph returned.
5. The response contains the answer plus the evidence: retrieved nodes, retrieved relationships, source documents and a retrieval-method line naming the traversal used.

Intent	Example question	Graph traversal used
explain_alert	Why was this alert created?	Event-[:CREATED_ALERT]->Alert-[:INDICATES]->Threat<-[:MITIGATES]-Control; Document-[:SUPPORTS_RESPONSE_TO]->Threat
controls_for_threat	What controls mitigate credential attacks?	Control-[:MITIGATES]->Threat
dependency_impact	Which services are affected if resource-service fails?	Service-[:DEPENDS_ON]->Service
users_failed_logins	Which users are linked to repeated failed logins?	User-[:TRIGGERED]->Event {failed_login_attempt}
users_high_severity_alerts	Which users are linked to high-severity alerts?	User-[:TRIGGERED]->Event-[:CREATED_ALERT]->Alert {HIGH}
threats_for_service	What threats target booking-service?	Threat-[:TARGETS]->Service
response_procedure	What response procedure should be followed?	Document-[:SUPPORTS_RESPONSE_TO]->Threat
alerts_affecting_service (default)	Show alerts affecting resource-service	Alert-[:AFFECTS]->Service

Evidence: the RAG interface

Question 1



Question 2

Ask a security question (RAG over the knowledge graph)

`POST /api/soc/ask`

Template-based GraphRAG: your question is matched to a Cypher query against Neo4j, and the answer is generated directly from the graph results — no LLM involved. Try: "Why was this alert created?", "What controls mitigate credential attacks?", "Which services are affected if resource-service fails?", "Which users are linked to repeated failed logins?"

Why was this alert created? | Ask

Answer: Alert alert-12039291-b7cb-49a9-805f-fb72ddd161c5 was created by rule "Abnormal Request Rate" (severity MEDIUM) because of 25 matching event(s) (evt-635a11c7-13a9-44a7-8dba-be6e9399c292, evt-5cf338c1-47ca-46fc-897c-22a30699c2fd, evt-0653d37d-92db-4901-a6e5-34a08be9c7c6, evt-057bd98c-e4b5-4ac1-bc8b-14cb51e17f9b, evt-09f1d42d-cfc4-422c-b4b3-3590feb50ebc, evt-fbc37530-c03c-403b-a721-631059b5be52, evt-2bf0a693-343b-4aac-87bd-46b0d74b6452, evt-2ca344dc-5a79-458f-88b4-ea8e4d77fba7, evt-cbbdcad9-1285-4644-8b3d-92a8496f852b, evt-16fdc6b3-28de-4a77-ae3e-0966bbd2d10e, evt-1c9f9391-f595-4b92-98e0-6a1339bc1985, evt-5ab089ee-12fa-490d-909c-a02a7648bc5a, evt-e7ccacdf-ce80-495d-8314-44421b0ed9e4, evt-e9d70066-f666-4163-bb8e-04f7e1972d7e, evt-71bb64df-481a-4e4d-a1f8-b9754252522f, evt-76d5d192-f457-4e0c-b6ea-3d0796538259, evt-74c9e388-580c-4fc8-b798-1234d3280b6f, evt-5a351053-144f-4948-a4cc-d6c15f8e0be2, evt-6fe2daac-7e5a-4e05-9195-dd279da65e1f, evt-0e067a6b-ae37-4cb9-b27c-178cd96f5951, evt-d0c88d4e-e1ab-48f3-ae48-47787fd85a11, evt-b5baf067-60d7-4779-8d19-63f5bd49f6a7, evt-e8f58ed8-7618-4e2a-a614-bc2f718caa61, evt-c46fca92-b466-49d6-90d0-9c0af19ee6ca, evt-ae8684cc-938c-4974-818a-d6ff730cb65a). This is linked to the threat "Denial of Service". Details: 25 requests to 'api-gateway' within 1 minute(s) - possible denial-of-service. Recommended controls are: Rate Limiting. Inspect the source IP distribution of recent requests; consider tightening the gateway's rate limiter for this route.

```
Intent: explain_alert
Retrieval method: Cypher traversal: Event -[:CREATED_ALERT]-> Alert -[:INDICATES]-> Threat <-[:MITIGATES]- Control, Document -
[:SUPPORTS_RESPONSE_TO]-> Threat
Retrieved nodes: Alert{alert-12039291-b7cb-49a9-805f-fb72ddd161c5}, Event{evt-635a11c7-13a9-44a7-8dba-be6e9399c292}, Event{evt-
5cf338c1-47ca-46fc-897c-22a30699c2fd}, Event{evt-0653d37d-92db-4901-a6e5-34a08be9c7c6}, Event{evt-057bd98c-e4b5-4ac1-bc8b-
14cb51e17f9b}, Event{evt-09f1d42d-cfc4-422c-b4b3-3590feb50ebc}, Event{evt-fbc37530-c03c-403b-a721-631059b5be52}, Event{evt-2bf0a693-
343b-4aac-87bd-46b0d74b6452}, Event{evt-2ca344dc-5a79-458f-88b4-ea8e4d77fba7}, Event{evt-cbbdcad9-1285-4644-8b3d-92a8496f852b},
Event{evt-16fdc6b3-28de-4a77-ae3e-0966bbd2d10e}, Event{evt-1c9f9391-f595-4b92-98e0-6a1339bc1985}, Event{evt-5ab089ee-12fa-490d-909c-
a02a7648bc5a}, Event{evt-e7ccacdf-ce80-495d-8314-44421b0ed9e4}, Event{evt-e9d70066-f666-4163-bb8e-04f7e1972d7e}, Event{evt-71bb64df-
481a-4e4d-a1f8-b9754252522f}, Event{evt-76d5d192-f457-4e0c-b6ea-3d0796538259}, Event{evt-74c9e388-580c-4fc8-b798-1234d3280b6f},
Event{evt-5a351053-144f-4948-a4cc-d6c15f8e0be2}, Event{evt-6fe2daac-7e5a-4e05-9195-dd279da65e1f}, Event{evt-0e067a6b-ae37-4cb9-b27c-
178cd96f5951}, Event{evt-d0c88d4e-e1ab-48f3-ae48-47787fd85a11}, Event{evt-b5baf067-60d7-4779-8d19-63f5bd49f6a7}, Event{evt-e8f58ed8-
7618-4e2a-a614-bc2f718caa61}, Event{evt-c46fca92-b466-49d6-90d0-9c0af19ee6ca}, Event{evt-ae8684cc-938c-4974-818a-d6ff730cb65a},
Threat{Denial of Service}, Control{Rate Limiting}
Retrieved relationships:
(Event evt-635a11c7-13a9-44a7-8dba-be6e9399c292)-[:CREATED_ALERT]->(Alert alert-12039291-b7cb-49a9-805f-fb72ddd161c5)
(Event evt-5cf338c1-47ca-46fc-897c-22a30699c2fd)-[:CREATED_ALERT]->(Alert alert-12039291-b7cb-49a9-805f-fb72ddd161c5)
(Event evt-0653d37d-92db-4901-a6e5-34a08be9c7c6)-[:CREATED_ALERT]->(Alert alert-12039291-b7cb-49a9-805f-fb72ddd161c5)
(Event evt-057bd98c-e4b5-4ac1-bc8b-14cb51e17f9b)-[:CREATED_ALERT]->(Alert alert-12039291-b7cb-49a9-805f-fb72ddd161c5)
(Event evt-09f1d42d-cfc4-422c-b4b3-3590feb50ebc)-[:CREATED_ALERT]->(Alert alert-12039291-b7cb-49a9-805f-fb72ddd161c5)
```

Question 3

Ask a security question (RAG over the knowledge graph)

`POST /api/soc/ask`

Template-based GraphRAG: your question is matched to a Cypher query against Neo4j, and the answer is generated directly from the graph results — no LLM involved. Try: "Why was this alert created?", "What controls mitigate credential attacks?", "Which services are affected if resource-service fails?", "Which users are linked to repeated failed logins?"

What controls mitigate credential attacks? | Ask

Answer: Credential Attack is mitigated by: Password Reset Procedure, Multi-Factor Authentication, Account Lockout.

```
Intent: controls_for_threat
Retrieval method: Cypher traversal: Control -[:MITIGATES]-> Threat
Retrieved nodes: Threat{Credential Attack}, Control{Password Reset Procedure}, Control{Multi-Factor Authentication}, Control{Account
Lockout}
Retrieved relationships:
(Control Password Reset Procedure)-[:MITIGATES]->(Threat Credential Attack)
(Control Multi-Factor Authentication)-[:MITIGATES]->(Threat Credential Attack)
(Control Account Lockout)-[:MITIGATES]->(Threat Credential Attack)
Source documents: (none)
```

8. Limitations and Design Choices

- Rules poll every 15 seconds rather than reacting to each event; a production SOC would stream events through a message queue and evaluate rules in real time.
- The RAG is template-based: it recognises the question types listed in section 7 by keywords and falls back to 'recent alerts' for open-ended questions. An LLM or vector layer (Option B) would handle freer questions. We chose template generation deliberately so every answer is fully traceable to graph evidence and the system runs offline without an API key.
- The Role, Asset, Incident and Vulnerability node types from the specification are not populated; the graph models the entities exercised by the booking-system scenario.
- The SOC endpoints (events, alerts, ask) are not JWT-protected; they are treated as an internal monitoring interface. A production deployment would restrict them to the SOC network or a service identity.
- Unauthorised-access alerts are grouped by source IP when no user is known; behind Docker, the IP shown is the bridge gateway (172.18.0.1), not the real client address.
- The Abnormal Request Rate rule is implemented and configurable, but an alert from it was not captured in the demonstration run

9. Evaluation Summary

Demonstrated workflow (end-to-end)

1. A user (Itumeleng) performed repeated failed login attempts through the gateway.
2. user-service recorded each as a failed_login_attempt event and published it to soc-service.
3. soc-service stored the events in socdb and wrote them to Neo4j as Event nodes linked to the User.
4. On its next 15-second pass the 'Multiple Failed Logins' rule matched five events for the same user within five minutes.
5. AlertService created a HIGH-severity alert (alert-aa43a2c9-...), stored in socdb and displayed on the SOC tab.
6. GraphWriter inserted the Alert into Neo4j and linked it to its events (CREATED_ALERT), the service (AFFECTS) and the Credential Attack threat (INDICATES).
7. The graph links that threat to its controls (MITIGATES) and the MITRE T1110 document.
8. The RAG question 'Why was this alert created?' retrieved this evidence and generated a grounded answer.

Test results

Test	Expected result	Actual result	Pass/Fail
Failed login rule	HIGH alert created after 5 failed logins for one user	alert-aa43a2c9-...: 'Multiple Failed Logins', HIGH, user Itumeleng, 5 related events	Pass

Unauthorised access rule	Alert created for repeated 401/403 / invalid-token requests	alert-b7b82fb5-... (MEDIUM, 3 events) and alert-a0d158ca-... (HIGH, 13 events)	Pass
Service failure rule	Alert created when a service is down	alert-dc6bf16b-...: 'Service Failure', HIGH, resource-service, 3 related events (resource-service container stopped)	Pass
Abnormal request rate rule	Alert created after >= 20 requests/min to a service	[UPDATE: alert id if captured, otherwise 'Not demonstrated']	[Pass]
Alert inserted into graph	Alert linked to user, events, threat and controls	Relationships created: User-TRIGGERED->Event-CREATED_ALERT->Alert-INDICATES->Threat<-MITIGATES-Control (Figure 3)	Pass
Graph queries	>= 3 queries return correct results	Six queries run in Neo4j Browser (section 6)	[Pass]
RAG explains alert	Answer uses graph evidence	[Answer text and evidence shown, see section 7 Question 1]	[Pass]
RAG controls question	Lists controls from the graph	[see section 7 Question 2]	[Pass]
RAG dependency question	Names booking-service as dependent on resource-service	[see section 7 Question 3]	[Pass]

.